

INSTITUTO FEDERAL DE MINAS GERAIS (IFMG)
CAMPUS BAMBUÍ
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Disciplina: BiSuEEA.512 – Sistemas Embarcados
Professor: Williams L. Nicomedes

Henrique Augusto G. Fernandes

LISTA DE EXERCÍCIOS Nº 1 — SISTEMAS EMBARCADOS

SUMÁRIO

1	QUESTÃO 1 – REFINAMENTO DA CLASSE MOTOR-INTERFACE . . .	3
1.1	Atributos da Classe	3
1.2	Operações da Classe	3
2	QUESTÃO 2 – INCORPORAÇÃO DE SWITCHES AO SISTEMA	5
2.1	Arquitetura da Solução Proposta	5
2.2	Protocolo de Comunicação	5
2.3	Implementação em Pseudocódigo	6
2.3.1	<i>Classe Switch-Controller</i>	6
2.3.2	<i>Formatação de Mensagens</i>	6
2.3.3	<i>Processamento no Console</i>	7
2.3.4	<i>Recepção e Execução na Pista</i>	8
2.4	Análise da Solução	8
3	CONSIDERAÇÕES FINAIS	10

1 QUESTÃO 1 – REFINAMENTO DA CLASSE MOTOR-INTERFACE

A classe *Motor-interface* é fundamental no sistema de ferromodelismo, pois é responsável pela conversão dos comandos de velocidade recebidos do controlador em sinais PWM que efetivamente controlam os motores dos trens. Seguindo o padrão de refinamento apresentado para as classes *Knobs* e *Panel*, procedeu-se ao detalhamento da *Motor-interface* considerando tanto seus atributos quanto suas operações.

1.1 Atributos da Classe

Para a implementação da funcionalidade PWM, identificou-se a necessidade dos seguintes atributos. Primeiramente, o *duty-cycle*, que representa a largura do pulso em percentual e varia de 0 a 100. Este valor é calculado a partir da velocidade recebida do controlador, originalmente codificada em 6 bits (0 a 63), através da seguinte proporção linear:

$$\text{duty-cycle} = \frac{\text{velocidade} \times 100}{63}$$

Por exemplo, quando o trem opera na velocidade máxima (63), o *duty-cycle* alcança 100%, enquanto na velocidade mínima (0), o *duty-cycle* é 0%.

Além do *duty-cycle*, considera-se necessário armazenar a frequência da onda PWM no atributo *pwm-frequency*, tipicamente configurado entre 1 kHz e 20 kHz para motores DC. Esta frequência determina quantos ciclos completos de PWM ocorrem por segundo e, geralmente, é mantida constante durante a operação do sistema.

A direção do movimento é controlada pelo atributo *direction*, uma variável booleana extraída do bit de direção presente no *speed-knob* (1 para frente, 0 para trás). Por fim, o atributo *pwm-state* mantém o estado instantâneo do sinal PWM (HIGH ou LOW), sendo atualizado constantemente durante a geração da onda.

1.2 Operações da Classe

As operações desta classe trabalham de forma coordenada para produzir o sinal PWM correto. A operação *calculate-duty-cycle(speed: integer)* recebe a velocidade (0–63) e retorna o *duty-cycle* correspondente em percentual. A operação *set-direction(dir: boolean)* atualiza o atributo de direção com base no bit recebido.

A operação central é *generate-pwm()*, que efetivamente produz a forma de onda PWM com base nos parâmetros configurados. Esta operação é invocada continuamente por um timer ou interrupção de hardware, alternando o *pwm-state* entre HIGH e LOW conforme necessário para manter o *duty-cycle* desejado.

A operação *update-motor-control(speed: integer, direction: boolean)* funciona como interface principal com o controlador, recebendo os comandos de velocidade e direção, atualizando os atributos internos e disparando a geração do PWM. Em situações de emergência, a operação *emergency-stop()* interrompe imediatamente a geração de PWM configurando o duty-cycle para 0, efetivamente parando o motor.

2 QUESTÃO 2 – INCORPORAÇÃO DE SWITCHES AO SISTEMA

Para a incorporação de um sistema de switches que permita controlar até 16 pontos de desvio na pista, cada um com ramificação em 2 trilhos, propõe-se a extensão do sistema existente com novas classes e modificações em algumas das classes já especificadas. A solução desenvolvida mantém a filosofia de design do sistema original, onde o console envia comandos que são formatados, transmitidos e executados pelos componentes apropriados.

2.1 Arquitetura da Solução Proposta

A modificação central envolve a criação de uma nova classe *Switch-Controller*, responsável pelo gerenciamento de todos os switches da pista. Esta classe armazena o estado de cada um dos 16 switches, onde cada switch pode estar em uma de duas posições (trilho A ou trilho B). Para tal, utiliza-se um array *switch-position[16]* do tipo booleano, onde 0 representa trilho A e 1 representa trilho B.

Considerando que cada switch requer apenas 1 bit de informação, o estado completo de todos os 16 switches pode ser codificado em apenas 16 bits (2 bytes). Adicionalmente, mantém-se um array *switch-status[16]* para monitoramento da operacionalidade de cada switch, aspecto importante para sistemas de segurança.

O painel de controle também necessita de extensões para permitir a interação do usuário com os switches. Na classe *Knobs*, propõe-se a adição de dois novos atributos: *switch-knob* (4 bits, valores de 0 a 15) para seleção do switch a ser configurado, e *switch-direction-knob* (1 bit) para escolha da posição desejada. Analogamente às modificações nos botões, a classe *Panel* recebe duas novas operações: *switch-number()*, que retorna o switch selecionado, e *switch-direction()*, que retorna a posição escolhida.

2.2 Protocolo de Comunicação

Um aspecto crítico da implementação refere-se à necessidade do sistema lidar com dois tipos diferentes de mensagens: comandos para trens e comandos para switches. O *Formatter* deve ser capaz de distinguir entre estes tipos. Para tal, adota-se um bit identificador no início de cada mensagem: 0 para comandos de trem e 1 para comandos de switch.

Assim, uma mensagem de controle de switch é composta por:

- 1 bit de tipo (sempre 1)
- 4 bits de identificação do switch (0–15)
- 1 bit de posição (A ou B)

- 2 bits de verificação de paridade

2.3 Implementação em Pseudocódigo

A implementação prática desta funcionalidade é ilustrada através do pseudocódigo que demonstra o fluxo completo desde a interação do usuário até a execução do comando no switch:

2.3.1 Classe *Switch-Controller*

```

CLASSE Switch-Controller:
  ATRIBUTOS:
    switch-position: ARRAY[16] DE BOOLEANO
    switch-status: ARRAY[16] DE BOOLEANO

  OPERAÇÕES:
    FUNÇÃO set-switch(id, position):
      SE (id válido E switch operacional) ENTÃO:
        switch-position[id] ← position
        acionar-atuador-fisico(id, position)
        registrar-log("Switch movido")
      SENÃO:
        reportar-erro("Switch inválido")
    FIM SE
  FIM FUNÇÃO

  FUNÇÃO get-switch-position(id): BOOLEANO
    SE (id >= 0 E id <= 15) ENTÃO:
      RETORNAR switch-position[id]
    SENÃO:
      RETORNAR -1 // Erro
    FIM SE
  FIM FUNÇÃO
FIM CLASSE

```

2.3.2 Formação de Mensagens

```

CLASSE Formatter (MODIFICADA):
  OPERAÇÕES:
    FUNÇÃO format-message(tipo, dados): BITS
      SE tipo = TIPO_TREM ENTÃO:

```

```

// Formato: 0 | 3 bits trem |
//           7 bits velocidade | 2 bits inércia
mensagem ← [0, dados.trem-id,
dados.velocidade, dados.inercia]
RETORNAR mensagem

SENÃO SE tipo = TIPO_SWITCH ENTÃO:
// Formato: 1 | 4 bits switch |
//           1 bit posição | 2 bits paridade
mensagem ← [1, dados.switch-id, dados.posicao]
paridade ← calcular-paridade(mensagem)
mensagem ← adicionar-bits(mensagem, paridade)
RETORNAR mensagem
FIM SE
FIM FUNÇÃO
FIM CLASSE

```

2.3.3 Processamento no Console

```

CLASSE Console (MODIFICADA):
OPERAÇÕES:
FUNÇÃO processar-entrada-usuario():
SE panel.panel-active() = VERDADEIRO ENTÃO:
// Detecta se está no modo trem ou switch
modo ← detectar-modo()

SE modo = MODO_TREM ENTÃO:
trem ← panel.train-number()
velocidade ← panel.speed()
inercia ← panel.inertia()

dados ← {trem, velocidade, inercia}
mensagem ← formatter.format-message(
TIPO_TREM, dados)
transmitter.send(mensagem)

SENÃO SE modo = MODO_SWITCH ENTÃO:
switch-id ← panel.switch-number()
posicao ← panel.switch-direction()

dados ← {switch-id, posicao}
mensagem ← formatter.format-message(

```

```

TIPO_SWITCH, dados)
transmitter.send(mensagem)
FIM SE
FIM SE
FIM FUNÇÃO
FIM CLASSE

```

2.3.4 Recepção e Execução na Pista

```

CLASSE Receiver (NO LADO DA PISTA):
OPERAÇÕES:
FUNÇÃO processar-mensagem(mensagem: BITS):
    tipo ← extrair-bit-tipo(mensagem)

    SE tipo = 0 ENTÃO: // Mensagem para trem
        trem-id ← extrair-bits(mensagem, 1, 3)
        velocidade ← extrair-bits(mensagem, 4, 10)
        inercia ← extrair-bits(mensagem, 11, 12)

        controller[trem-id].set-speed(velocidade)
        controller[trem-id].set-inertia(inercia)

    SENÃO SE tipo = 1 ENTÃO: // Mensagem para switch
        switch-id ← extrair-bits(mensagem, 1, 4)
        posicao ← extrair-bit(mensagem, 5)
        paridade ← extrair-bits(mensagem, 6, 7)

        SE verificar-paridade(mensagem, paridade) ENTÃO:
            switch-controller.set-switch(
                switch-id, posicao)
        SENÃO:
            reportar-erro-comunicacao()
    FIM SE
FIM SE
FIM FUNÇÃO
FIM CLASSE

```

2.4 Análise da Solução

O pseudocódigo apresentado demonstra o fluxo completo desde a interação do usuário até a execução do comando no switch. O usuário seleciona o modo

de operação (trem ou switch), escolhe qual switch configurar através do *switch-knob*, define a posição desejada com o *switch-direction-knob*, e o sistema automaticamente formata e transmite o comando apropriado.

No lado receptor, o *Receiver* decodifica a mensagem e aciona o *Switch-Controller*, que por sua vez move fisicamente o switch para a posição desejada. Destaca-se que a implementação de verificação de paridade nas mensagens de switch é particularmente importante, pois um erro de comunicação que cause o acionamento incorreto de um switch poderia resultar em colisões ou descarrilamentos.

O sistema também mantém um registro do estado de cada switch, permitindo operações de consulta e diagnóstico, além de garantir que switches defeituosos não sejam acionados. A verificação de paridade mostra-se essencial para evitar acionamentos incorretos que poderiam causar acidentes na operação do sistema.

3 CONSIDERAÇÕES FINAIS

O presente exercício demonstra que a especificação de sistemas embarcados constitui um processo complexo que exige atenção a múltiplos detalhes simultâneos. Ao detalhar a classe *Motor-interface*, verificou-se a necessidade de considerar não apenas os parâmetros básicos do PWM, mas também questões como conversão de valores, tratamento de direção e resposta a emergências.

Similarmente, a incorporação dos switches revelou a necessidade de entender não apenas uma classe, mas todo o ecossistema do sistema, incluindo protocolos de comunicação, formatação de mensagens, verificação de erros e tratamento de falhas. Este processo de especificação permite antecipar situações que poderiam passar despercebidas em uma implementação direta, contribuindo significativamente para a robustez e confiabilidade do sistema final.

A utilização de pseudocódigo estruturado mostrou-se adequada para visualizar claramente o fluxo de controle e as interações entre os componentes, facilitando tanto a compreensão do sistema quanto sua posterior implementação em linguagens de programação específicas para sistemas embarcados. Observa-se que a metodologia de especificação orientada a objetos, combinada com protocolos de comunicação bem definidos e mecanismos de verificação de integridade, constitui uma base sólida para o desenvolvimento de sistemas embarcados confiáveis e seguros.